

ClassTrack — Classmate Performance & Challenge Tracker

Technical Design Document

- 1. Overview
- 2. User Roles
- 3. Core Features
 - 3.1 Challenges
 - 3.2 Daily Task Submission
 - 3.3 AI Article Grading
 - 3.4 Scoring & Ranking
 - 3.5 Member Dashboard
 - 3.6 Leaderboard
 - 3.7 Job Application Tracker (per member)
 - 3.8 WhatsApp Notifications (design only, per your request)
- 4. Data Model
- 5. System Architecture
- 6. Key API Endpoints (draft)
- 7. AI Grading Rubric (draft, tune as needed)
- 8. Notification Logic (pseudocode)
- 9. Suggested Tech Stack
- 10. Build Roadmap (MVP-first)
- 11. Open Questions to Settle Before Building

ClassTrack — Classmate Performance & Challenge Tracker

Technical Design Document

1. Overview

ClassTrack is a web app for a study group (e.g., “100 Days of Python”) to track daily task completion, rank members, grade submitted articles with AI, and nudge lagging students via WhatsApp.

Core loop: Admin creates a challenge → members log daily tasks (code snippets or article links) → system scores activity → leaderboard updates → students falling behind get a WhatsApp nudge.

2. User Roles

Role	Capabilities
Admin (you)	Create/manage challenges, view all members, configure scoring rules, trigger/schedule notifications, override grades
Member	Join challenge(s), submit daily tasks (code paste or article link), view own streak/history, view leaderboard
System (AI grader)	Fetches article link, scores it against a rubric, returns feedback

3. Core Features

3.1 Challenges

- A challenge has a name, start/end date, target (e.g. “100 days”), and task types allowed (code / article / both).
- A member can be enrolled in multiple challenges.
- Each calendar day within the challenge window is a “slot” a member can fill with a submission.

3.2 Daily Task Submission

- **Code submission:** paste code + select language + tag which “day” it counts for. Optional: paste a problem link (LeetCode/HackerRank/SQL playground) for reference.
- **Article submission:** paste a URL (Medium, dev.to, personal blog, LinkedIn article).
- One submission per member per day per challenge counts toward the streak (configurable — you may allow multiple but only count the first for streak purposes).

3.3 AI Article Grading

- On article submission, backend fetches the URL content, strips boilerplate, and sends it to an LLM with a rubric (e.g., clarity, technical accuracy, depth, originality) → returns a score (0-10) + short feedback.
- Score contributes to the member’s points; feedback is shown to the member, not just a number.
- Code submissions can optionally get a lightweight AI review too (style/correctness comments) — separate from the streak count.

3.4 Scoring & Ranking

Suggested point model (fully configurable):

Action	Points
Daily task completed (code)	10
Daily task completed (article)	15 (higher effort)
Article AI-grade bonus	+ up to 10 (scaled from AI score)
Streak bonus	+5 for every 7 consecutive days
Missed day	0 (no penalty, but breaks streak)

Leaderboard = sum of points within a challenge window (and optionally an all-time leaderboard across challenges). Ties broken by longest current streak.

3.5 Member Dashboard

- Calendar heatmap (like GitHub’s contribution graph) showing which days were completed this month.
- Current streak, longest streak, total points, rank.
- List of past submissions with AI feedback.
- Job Application Tracker widget (log/status board) — see 3.7.

3.6 Leaderboard

- Per-challenge leaderboard, refreshed on each submission (or via scheduled recompute).
- Global/all-time leaderboard across all challenges a member has joined.

3.7 Job Application Tracker (per member)

- A personal section on each member’s profile — separate from challenge scoring, purely for their own accountability as they job-hunt.
- Fields per entry: company, role, date applied, status (Applied / OA / Interview / Offer / Rejected), source (LinkedIn, referral, company site), notes/follow-up date.
- Simple stats: total applications this month, response rate, interviews landed — visible only to the member (and admin, if the group wants shared visibility for mutual support/referrals).

- Optional: a lightweight group-wide “who got an interview/offer this week” shout-out feed, since this is a strong peer-motivation signal for a job-seeking cohort — but keep it opt-in per entry, not automatic, since not everyone wants every application visible.

3.8 WhatsApp Notifications (design only, per your request)

- A scheduled job (e.g., daily at 8pm) checks each active member’s completion status for the day/week.
- If a member has missed **N consecutive days** (configurable threshold, e.g. 2), they’re flagged as “lagging.”
- A message is sent via WhatsApp Business API summarizing: days missed, current streak lost, and a specific nudge (e.g., “You’re 3 days behind on 100 Days of Python — submit today’s task to keep pace with the group average”).
- Admin also gets a weekly digest of who’s lagging.

4. Data Model

User

└ id, name, phone_number (for WhatsApp), email, joined_at

Challenge

└ id, name, description, start_date, end_date, task_types_allowed, scoring_rules (JSON)

Enrollment

└ id, user_id (FK), challenge_id (FK), joined_at

Submission

└ id, enrollment_id (FK), day_number, submission_type (code | article)
└ content (code text OR article URL)
└ language (nullable, for code)
└ ai_score (nullable), ai_feedback (nullable, text)
└ points_awarded, submitted_at

Streak (derived/cached table, or computed on the fly)

└ enrollment_id (FK), current_streak, longest_streak, last_submission_date

Notification

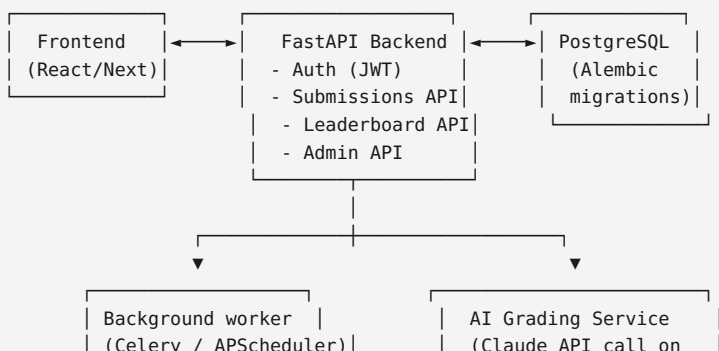
└ id, user_id (FK), challenge_id (FK), type (lagging_alert | weekly_digest)
└ message_sent, sent_at, status (sent | failed)

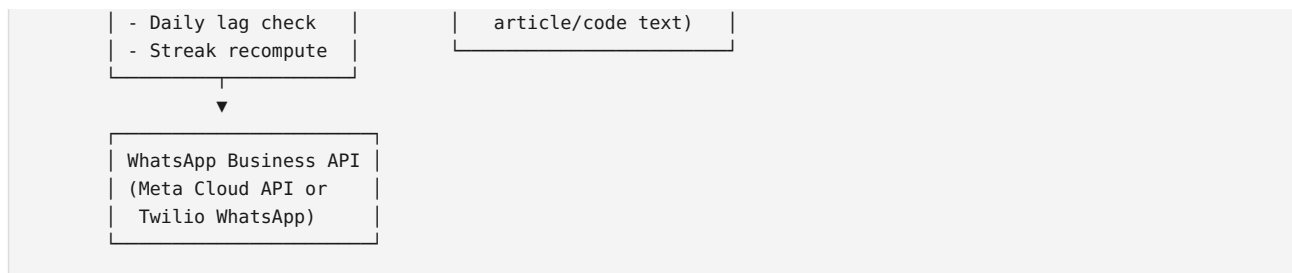
JobApplication

└ id, user_id (FK), company, role, date_applied
└ status (applied | oa | interview | offer | rejected)
└ source (linkedin | referral | company_site | other)
└ notes, follow_up_date, is_public (bool, for the opt-in shout-out feed)

This maps cleanly to a normalized relational schema — a good fit for PostgreSQL, consistent with the stack you’re already using on AddressYangu.

5. System Architecture





Why this shape: - FastAPI handles synchronous request/response (submissions, leaderboard reads, auth) — same stack you’re already building fluency in. - A background worker (APScheduler is enough at this scale; Celery + Redis if you outgrow it) handles anything time-based: nightly lag checks, streak recalculation, digest emails/messages. This decouples “a student submitted a task” from “check if anyone needs a nudge,” which should never block the request path. - AI grading is called asynchronously after an article submission (don’t make the user wait on an LLM call in the request-response cycle) — queue it, grade it, update the record, and the frontend polls or the UI shows “grading...” until it’s ready. - WhatsApp integration sits behind the worker, not the API — nudges are always triggered by the scheduler, never inline with a user action.

6. Key API Endpoints (draft)

Method	Endpoint	Purpose
POST	/auth/login	Member/admin login
POST	/challenges	Admin creates a challenge
POST	/challenges/{id}/enroll	Member joins a challenge
POST	/submissions	Submit a task (code or article link)
GET	/submissions/me?challenge_id=	My submission history
GET	/leaderboard/{challenge_id}	Ranked list for a challenge
GET	/leaderboard/global	All-time ranked list
GET	/members/{id}/dashboard	Streak, heatmap, points for one member
POST	/admin/notify/run	Manually trigger the lag-check job (for testing)
POST	/job-applications	Log a new job application
GET	/job-applications/me	My job application history + stats
PATCH	/job-applications/{id}	Update status (e.g. Applied → Interview)
GET	/job-applications/feed	Opt-in group shout-out feed (interviews/offers)

7. AI Grading Rubric (draft, tune as needed)

For article submissions, prompt the model with something like:

- **Clarity (0-3):** Is the explanation easy to follow for someone learning the topic?
- **Technical accuracy (0-3):** Are claims/code examples correct?
- **Depth (0-2):** Does it go beyond a surface-level tutorial recap?
- **Originality (0-2):** Own examples/insights vs. rehashing docs?

Return a structured JSON response (score breakdown + 2-3 sentences of feedback) so it’s easy to store and display — this also keeps grading consistent across submissions rather than free-form prose each time.

8. Notification Logic (pseudocode)

```
for each active enrollment:
    last_day = last_submission_date(enrollment)
    days_missed = (today - last_day).days - 1 # excluding today
    if days_missed >= LAG_THRESHOLD:
        message = build_message(
            name=user.name,
            challenge=challenge.name,
            days_missed=days_missed,
            streak_lost=previous_streak,
            group_avg_points=get_group_average(challenge)
        )
        send_whatsapp(user.phone_number, message)
        log_notification(user, challenge, "lagging_alert")
```

Run this once daily via the scheduler. Keep `LAG_THRESHOLD` and message wording configurable so you can tune tone (motivational vs. blunt) without a redeploy.

9. Suggested Tech Stack

Layer	Choice	Why
Backend	FastAPI	You're already building this fluency for AddressYangu
DB	PostgreSQL + Alembic	Same as your current stack; relational fits this data model well
Frontend	React (or plain HTML/HTMX if you want to move faster)	Leaderboard/dashboard needs interactivity but nothing exotic
Background jobs	APScheduler (start simple) → Celery + Redis if scale demands	Matches MVP-first approach
AI grading	Claude API (structured JSON output)	Consistent, explainable scoring
WhatsApp	Meta WhatsApp Cloud API (free tier, official) or Twilio (easier DX, paid)	Meta's Cloud API is cheaper long-term for a small group
Hosting	Railway/Render (backend + Postgres) for MVP	Low-friction deploy while validating

10. Build Roadmap (MVP-first)

- Phase 1 — Core tracking:** Users, challenges, enrollment, manual submissions (code paste), streak calculation, basic leaderboard. No AI, no WhatsApp yet.
- Phase 2 — Dashboard polish:** Calendar heatmap, per-member stats, admin challenge management UI.
- Phase 3 — AI grading:** Article URL submission → fetch content → Claude API scoring → store + display feedback.
- Phase 4 — WhatsApp nudges:** Scheduler job, Meta Cloud API integration, lag-detection logic, admin digest.
- Phase 5 — Refinement:** Configurable scoring rules per challenge, global leaderboard across challenges, gamification extras (badges, weekly MVP).

Each phase is independently demoable to your classmates, so you get feedback before over-building.

11. Open Questions to Settle Before Building

- Should missed days carry any penalty, or just break the streak (no negative points)?
- Is there one global challenge at a time, or can multiple run concurrently (e.g. “100 Days of Python” + “SQL Sprint”)?
- Who verifies code submissions aren’t copy-pasted from someone else — spot checks, or trust-based for a peer group?
- WhatsApp opt-in: since it needs phone numbers and Meta’s Business API requires template-approved messages for the first outbound contact in a 24h window, plan for members to message the bot number once to open the conversation window.